Operator warunkowy – PHP 2020

Operator warunkowy pozwala na ustalenie wartości wyrażenia w zależności od prawdziwości danego warunku. Ma on postać:

warunek ? *wartość1* : *wartość2*

Oznacza to: jeśli warunek jest prawdziwy, podstaw za wartość całego wyrażenia *wartość1*, w przeciwnym wypadku za wartość wyrażenia podstaw *wartość2*.

```
<?php
$liczba = 100;
$wynik = ($liczba < 0) ? -1 : 1;
echo $wynik;
?>
```

Operator warunkowy ma mniejszy priorytet niż operatory relacyjne. W związku z tym nawias okrągły, wprowadzony do wyrażenia w celu zwiększenia czytelności zapisu, może zostać pominięty i wyrażenie może mieć również postać:

```
$wynik = $liczba < 0 ? -1 : 1;
```

Typ string - opis

Typ string to typ łańcuchowy, który służy do zapamiętywania sekwencji znaków. Nie ma ograniczenia długości tego ciągu. Do zapamiętywania pojedynczego znaku jest wykorzystywany jeden bajt.

Łańcuch znaków można utworzyć na cztery sposoby:

- używając znaków apostrofu,
- używając znaków cudzysłowu,
- używając składni heredoc,
- używając składni newdoc.

Wykorzystanie znaków apostrofu

Pierwszym sposobem deklaracji łańcucha znakowego jest ujęcie go w znaki apostrofu. PHP praktycznie nie dokonuje interpretacji takiego ciągu znaków, jeśli zatem naprzykład wyświetlamy go na ekranie, pojawi się w większości przypadków w niezmienionej postaci. Przykładem takiego ciągu jest 'abc'. Wyjątkiem od tej zasady jest specjalne traktowanie samego znaku apostrofu. Jeśli chcemy uzyskać go w napisie, koniecznie musimy poprzedzić go znakiem \ (lewy ukośnik, z ang. *backslash*). Podobnie jeżeli chcemy uzyskać sekwencję \', należy napisać \\'. Ta zasada dotyczy także samego znaku ukośnika.

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-2">
<title>Mojastrona WWW</title>
</head>
<body>
<div>
<?php
echo 'To jest znak apostrofu: \";
echo '<br />';
echo 'To jest sekwencja znaków ukośnik-apostrof: \\\";
?>
</div>
</body>
</html>
```

Wynik:

To jest znak apostrofu: '

To jest sekwencja znaków ukośnik-apostrof: \'

Wykorzystanie znaków cudzysłowu

Drugim sposobem deklaracji łańcucha znakowego jest ujęcie go w znaki cudzysłowu. Przykładem takiego ciągu jest "abc". Takie ciągi są przetwarzane przez PHP. Najważniejsze jest to, że jeśli w tego typu sekwencji znajdzie się określenie zmiennej, zostanie ono zmienione na wartość przez nią.

Wykorzystanie składni heredoc

W przypadku składni heredoc łańcuch znakowy należy rozpocząć od sekwencji <<<, po której musi nastąpić identyfikator. Tego identyfikatora należy następnie użyć w celu zasygnalizowania końca łańcucha znakowego. Dla nazwy identyfikatora obowiązującatakie same zasady jak przy nazewnictwie zmiennych. Może się ona zaczynać wyłącznie od znaku podkreślenia lub litery i może zawierać dowolną kombinację liter, cyfr i znaków podkreślenia. Linia kończąca nie może natomiast zawierać żadnych innych znaków oprócz identyfikatora i średnika. Uwaga ta dotyczy wszystkich znaków, również spacji, tabulatorów itp.

```
$napis = <<<ID1
Tutaj rozpoczyna się napis
ID1;
```

Zmienne zawarte w ciągu znaków korzystającym z opisywanej składni, podobnie jak w przypadku składni z cudzysłowami, zostaną zamienione na odpowiadające im wartości.

Wykorzystanie składni heredoc do utworzenia łańcucha znakowego

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-2">
<title>Mojastrona WWW</title>
</head>
<body>
<div>
<?php
$napis = <<<ID1
Tutaj rozpoczyna się napis.<br />
Składa się on z kilku linii.<br />
Dlatego też została wykorzystana składnia heredoc.<br />
ID1;
echo $napis.”abc”;
?>
</div>
</body>
</html>
```

Wynik:

```
Tutaj rozpoczyna się napis.
Składa się on z kilku linii.
Dlatego też została wykorzystana składnia heredoc.
```

Wykorzystanie składni newdoc

Składnia newdoc jest bardzo podobna do heredoc. Różnica jest taka, że w przypadku newdoc ciąg nie jest interpretowany, a zatem nazwy zmiennych nie są zamieniane na odpowiadające im wartości. Zatem różnica między hewdoc a heredoc jest taka jak między użyciem znaków apostrofu i cudzysłowu. Składnia newdoc została wprowadzona do PHP w wersji 5.3.0.

```
$napis = <<<'ID2'
Tutaj rozpoczyna się napis
ID2;
```

Rodzaje ciągów znaków

```
<?php
$liczba = 100;
$ciag = "abcd";
$str1 = "Zmienna liczba = $liczba, a zmienna ciag = $ciag.";
$str2 = 'Zmienna liczba = $liczba, a zmienna ciag = $ciag.';
$str3 = <<<ID1
Zmienna liczba = $liczba,
a zmienna ciag = $ciag.
ID1;
$str4 = <<<'ID2'
Zmienna liczba = $liczba,
a zmienna ciag = $ciag.
ID2;
echo "Zmienna str1 (składnia ze znakami cudzysłowu):<br />";
echo $str1;
echo "<br /><br />";
echo "Zmienna str2 (składnia ze znakami apostrofu):<br />";
echo $str2;
echo "<br /><br />";
echo "Zmienna str3 (składnia heredoc):<br />";
echo $str3;
echo "<br /><br />";
echo "Zmienna str4 (składnia nowdoc):<br />";
echo $str4;
echo "<br /><br />";
?>
```

Wynik:

Zmienna str1 (składnia ze znakami cudzysłowu):
Zmienna liczba = 100, a zmienna ciag = abcd.

Zmienna str2 (składnia ze znakami apostrofu):
Zmienna liczba = $liczba, a zmienna ciag = $ciag.

Zmienna str3 (składnia heredoc):
Zmienna liczba = 100, a zmienna ciag = abcd.

Zmienna str4 (składnia nowdoc):
Zmienna liczba = $liczba, a zmienna ciag = $ciag.

Operator (kropka) jest operatorem łączenia ciągów – a nie +.

```
$str1 = "abc";
$str2 = "def";
$str3 = $str1 + $str2;
```

Zmienna str3 będzie zawierała wartość 0. typu integer. Otóż ponieważ operator + działa na argumentach arytmetycznych, a zmienne $str1 i $str2 są typu string (są łańcuchami znakowymi, napisami), przed wykonaniem dodawania nastąpi konwersja argumentów do typu arytmetycznego.

Jeśli mamy zmienną $liczba o wartości 1.4, to zapis:

```
echo "abc $liczba def";
```

zadziała zgodnie z przewidywaniami, podobnie jak:

```
echo "abc$liczba def";
```

W pierwszym przypadku na ekranie pojawi się napis abc 1.4 def, a w drugim — abc1.4def.

Nie zadziała natomiast instrukcja w postaci: echo "abc$liczbadef";
gdyż PHP będzie wtedy szukało zmiennej o nazwie $liczbadef, której nie ma w skrypcie.

To typowa czynność często wykonywana przy programowaniu w PHP na potrzeby WWW. To zadanie może być wykonane na kilka sposobów.

1. ze znakami cudzysłowu.

   Instrukcja echo przyjmie wtedy postać:

```
echo "pierwsza: $zmienna1 druga: $zmienna2 trzecia: $zmienna3 koniec";
```

Wartości zmiennych zostaną zamienione na odpowiadające im wartości, bowiem PHP dokona interpretacji takiego ciągu.

2. ręczne składanie wartości całego ciągu za pomocą operatora łączenia łańcuchów.

```
echo 'pierwsza: '.$zmienna1.' druga: '.$zmienna2.' trzecia: '.$zmienna3.' koniec ';
```

W tym przypadku zmienne będą konwertowane do typu string i doklejane do pozostałych fragmentów ciągu.

3. oddzielenie poszczególnych składowych instrukcji echo za pomocą przecinków.

```
echo 'pierwsza: ', $zmienna1, ' druga: ', $zmienna2, ' trzecia: ', $zmienna3,' koniec ';
```

4.skorzystanie ze składni heredoc

```
<?php
$zmienna1 = 1.7894321;
$zmienna2 = 154643212;
$zmienna3 = 'abcdEFGH';

$liczba = 100;
$str = <<<ID
pierwsza: $zmienna1 druga: $zmienna2 trzecia: $zmienna3 koniec
ID;

echo $str;
?>
```

Wynik:

```
pierwsza: 1.7894321 druga: 154643212 trzecia: abcdEFGH koniec
```

zmiany wielkości liter w ciągach

Nazwa funkcji	Opis	Przykład wywołania
`strtolower`	Przetwarza ciąg przekazany w postaci argumentu w taki sposób, że wszystkie litery zostaną zamienione na małe. Zwraca ciąg wynikowy.	`$str = strtolower($str)`
`strtoupper`	Przetwarza ciąg przekazany w postaci argumentu w taki sposób, że wszystkie litery zostaną zamienione na wielkie. Zwraca ciąg wynikowy.	`$str = strtoupper($str)`
`ucfirst`	Przetwarza ciąg przekazany w postaci argumentu w taki sposób, że pierwsza litera zostanie zamieniona na wielką. Zwraca ciąg wynikowy.	`$str = ucfirst($str)`
`ucwords`	Przetwarza ciąg przekazany w postaci argumentu w taki sposób, że wszystkie pierwsze litery wyrazów zostaną zamienione na wielkie. Zwraca ciąg wynikowy.	`$str = ucwords($str)`

Wykorzystanie funkcji zmieniających wielkość liter

```
<?php
$str = "przykładowy CIĄG zNaKóW";
echo "Ciąg oryginalny: ";
echo $str;
echo "<br />Po użyciu funkcji strtolower: ";
echo strtolower($str);
echo "<br />Po użyciu funkcji strtoupper: ";
echo strtoupper($str);
echo "<br />Po użyciu funkcji ucfirst: ";
echo ucfirst($str);
echo "<br />Po użyciu funkcji ucwords: ";
echo ucwords($str);
?>
```

funkcje usuwające z początków i końców ciągów tzw. białe znaki.

Użyteczne funkcje to trim, ltrim i rtrim. Usuwają one następujące znaki:

- " " — znak spacji o kodzie 32 (0x20 szesnastkowo),
- \t — znak tabulacji o kodzie 9,
- \n — znak nowej linii o kodzie 10 (0x0A szesnastkowo),
- \r — znak powrotu karetki (z ang. *carriage return*) o kodzie 13 (0x0D szesnastkowo),
- \0 — znak o kodzie 0,
- \0x0B — znak tabulacji pionowej o kodzie 11.

Funkcja trim usuwa wymienione znaki z początku i z końca ciągu, funkcja ltrim—tylko z początku, a funkcja rtrim— tylko z końca. Zamiast rtrim można również użyć nazwy chop, co ma identyczne znaczenie. Wszystkie funkcje przyjmują jako argument ciąg do przetworzenia i zwracają ciąg przetworzony. W przypadku gdy chcemy usuwaćinne znaki niż wymienione wyżej, należy zastosować drugi parametr wywołania. Taka konstrukcja ma następującą postać:

nazwa_funkcji("*ciąg_przetwarzany*", "*znaki_do_usunięcia*");

```
<?php
$str = " ... _Przykładowy ciąg_ ... ";
echo "Pierwotna zawartość ciągu str:<br />";
echo "-$str-";
$str = ltrim($str);
echo "<br>Po wywołaniu ltrim(\$str):<br />";
echo "-$str-";
$str = rtrim($str);
echo "<br />Po wywołaniu rtrim(\$str):<br />";
echo "-$str-";
$str = trim($str, "_. ");
echo "<br />Po wywołaniu trim(\$str, \"_. \"):<br />";
echo "-$str-";
?>
```

Wynik: // http://phpfiddle.org/ lub http://phptester.net/ uwaga!!

```
Pierwotna zawartość ciągu str:
- ... _Przykładowy ciąg_ ... -
Po wywołaniu ltrim($str):
-... _Przykładowy ciąg_ ... -
Po wywołaniu rtrim($str):
-... _Przykładowy ciąg_ ...-
Po wywołaniu trim($str, "_. "):
-Przykładowy ciąg-
```

Przetwarzanie ciągów znaków

funkcja zamieniające podciągi: str_replace, str_ireplace, substr_replace lub strtr.

(PHP 4, PHP 5, PHP 7)
str_replace - Zastąp wszystkie wystąpienia szukanego ciągu ciągiem zastępczym

Opis
mieszane **str_replace** (mieszane $search , mieszane $replace , mieszane $subje ct [, int &$count])

Ta funkcja zwraca ciąg lub tablicę ze wszystkimi wystąpieniami search in subject zamienionymi na podaną replace wartość.

Jeśli nie potrzebujesz wymyślnych reguł zastępowania (takich jak wyrażenia regularne), powinieneś zawsze używać tej funkcji zamiast preg_replace () .

```
<?php
$zdanie = "Przykładowe Pierwsze zdanie";
echo  str_replace("pierwsze", "drugie", $zdanie)."<br>";
// W zdaniu słowo "piersze" zostanie zamienione słowem "drugie" w rezultacie otrzymamy: Przykładowe drugie zdanie

$zdanie2=str_replace("pierwsze", "drugie", $zdanie);
echo $zdanie."<br>";
echo $zdanie2."<br>";


?>
```

str_ireplace()

Nie odróżniająca wielkość znaków odmiana funkcji str_replace ().

```
<?php
$zdanie = "Przykładowe Pierwsze zdanie";
echo  str_ireplace("pierwsze", "Drugie", $zdanie)."<br>";
// W zdaniu słowo "piersze" zostanie zamienione słowem "drugie" w rezultacie otrzymamy: Przykładowe drugie zdanie
```

```
$zdanie2=str_ireplace("Pierwsze", "drugie", $zdanie);
echo $zdanie."<br>";
echo $zdanie2."<br>";


$zdanie2=str_ireplace("PierwszE", "drugie", $zdanie);
echo $zdanie."<br>";
echo $zdanie2."<br>";
?>
```

strchr()
Alias dla strstr().

---

stristr()
Odróżniająca wielkość znaków wersja funkcji strstr().

---

strstr()
Wyszukuje jednego lub wielu znaków w ciagu, po czym usuwa to co było, przed znalezionym rekordem, a wyświetla pozostałość łącznie z szukanym znakiem.

SPIS FUNKCJI PHP – STRING

Źródło:
http://man.hubwiz.com/docset/PHP.docset/Contents/Resources/Documents/php.net/manual/en/ref.strings.html

addcslashes - Cytuj ciąg z ukośnikami w stylu C.
addlashes - Cytuj ciąg z ukośnikami
bin2hex - Konwertuj dane binarne na reprezentację szesnastkową
chop - alias rtrim
chr - Zwraca określony znak
chunk_split - Podziel ciąg na mniejsze części
convert_cyr_string - Konwertuj z jednego zestawu znaków cyrylicy na inny
convert_uudecode - Dekoduj ciąg znaków uuencoded
convert_uuencode - Uuencode ciąg
count_chars - Zwraca informacje o znakach użytych w ciągu
crc32 - oblicza wielomian crc32 łańcucha
crypt - Jednokierunkowe mieszanie ciągów znaków
echo - Wyświetla jeden lub więcej ciągów

eksplodować - Podziel ciąg przez ciąg
fprintf - Napisz sformatowany ciąg do strumienia
get_html_translation_table - zwraca tabelę tłumaczeń używaną przez htmlspecialchars i htmlentities
hebrev - Konwertuj logiczny tekst hebrajski na tekst wizualny
hebrevc - Konwertuj logiczny tekst hebrajski na tekst wizualny z konwersją do nowej linii
hex2bin - dekoduje ciąg binarny zakodowany szesnastkowo
html_entity_decode - Konwertuj wszystkie encje HTML na odpowiednie znaki
htmlentities - Konwertuj wszystkie odpowiednie znaki na encje HTML
htmlspecialchars_decode - Konwertuj specjalne encje HTML z powrotem na znaki
htmlspecialchars - Konwertuj znaki specjalne na encje HTML
implode - Połącz elementy tablicy za pomocą ciągu
join - alias implode
lcfirst - Tworzy małą literę pierwszego znaku w ciągu
levenshtein - Oblicz odległość Levenshteina między dwoma strunami
localeconv - Uzyskaj informacje o formatowaniu liczb
ltrim - Usuwa białe znaki (lub inne znaki) z początku ciągu
md5_file - oblicza skrót md5 podanego pliku
md5 - Oblicz wartość skrótu md5 ciągu
metaphone - Oblicz klucz metafonu dla ciągu
money_format - formatuje liczbę jako ciąg waluty
nl_langinfo - Zapytanie o język i informacje o lokalizacji
nl2br - Wstawia znaki końca wiersza HTML przed wszystkimi znakami nowej linii w ciągu
number_format - Sformatuj liczbę za pomocą zgrupowanych tysięcy
ord - Zwraca wartość znaku ASCII
parse_str - analizuje ciąg na zmienne
print - Wyświetla ciąg
printf - Wyprowadza sformatowany ciąg
quoted_printable_decode - Konwertuje napis w cudzysłowie do wydrukowania na 8-bitowy
quoted_printable_encode - Konwertuje 8-bitowy ciąg na ciąg do wydrukowania w cudzysłowie
quotemeta - Cytuj meta znaki
rtrim - Usuwa białe znaki (lub inne znaki) z końca łańcucha
setlocale - Ustaw informacje o lokalizacji
sha1_file - Oblicz hash sha1 pliku
sha1 - Oblicz skrót sha1 łańcucha
similar_text - Oblicz podobieństwo między dwoma ciągami
soundex - Oblicz klucz soundex łańcucha
sprintf - Zwraca sformatowany ciąg

sscanf - analizuje dane wejściowe z ciągu zgodnie z formatem
str_getcsv - Analizuje ciąg CSV w tablicy
str_ireplace - wersja str_replace bez rozróżniania wielkości liter.
str_pad - Dopełnij ciąg do określonej długości innym ciągiem
str_repeat - Powtórz ciąg
str_replace - Zastąp wszystkie wystąpienia szukanego ciągu ciągiem zastępczym
str_rot13 - Wykonaj transformację rot13 na strunie
str_shuffle - Losowo tasuje ciąg
str_split - Konwertuj ciąg na tablicę
str_word_count - Zwraca informacje o słowach użytych w ciągu
strcasecmp - Binarne bezpieczne porównanie ciągów bez rozróżniania wielkości liter
strchr - Alias of strstr
strcmp - Porównanie bezpiecznych łańcuchów binarnych
strcoll - Porównanie ciągów w oparciu o ustawienia regionalne
strcspn - Znajdź długość początkowego segmentu nie pasującą do maski
strip_tags - Usuń znaczniki HTML i PHP z ciągu znaków
stripcslashes - Cytuj ciąg cytowany za pomocą addcslashes
stripos - Znajdź pozycję pierwszego wystąpienia podciągu bez rozróżniania wielkości liter w ciągu
stripslashes - Cytuje ciąg w cudzysłowie
stristr - strstr
strlen - Uzyskaj długość ciągu
strnatcasecmp - Porównania ciągów bez rozróżniania wielkości liter przy użyciu algorytmu „porządku naturalnego”
strnatcmp - Porównania ciągów przy użyciu algorytmu „porządku naturalnego”
strncasecmp - Porównanie ciągów pierwszych n znaków bez rozróżniania wielkości liter
strncmp - Binarne porównanie bezpiecznych ciągów pierwszych n znaków
strpbrk - Wyszukaj ciąg znaków dla dowolnego zestawu znaków
strpos - Znajdź pozycję pierwszego wystąpienia podciągu w ciągu
strrchr - Znajdź ostatnie wystąpienie znaku w ciągu
strrev - Odwróć ciąg
strripos - Znajdź pozycję ostatniego wystąpienia podciągu bez rozróżniania wielkości liter w ciągu
strrpos - Znajdź pozycję ostatniego wystąpienia podciągu w ciągu
strspn - Znajduje długość początkowego segmentu ciągu składającego się wyłącznie ze znaków zawartych w danej masce.
strstr - Znajdź pierwsze wystąpienie ciągu
strtok - łańcuch tokenizacji
strtolower - Utwórz małe litery w ciągu
strtoupper - tworzy wielkie litery w łańcuchu
strtr - Tłumacz znaki lub zamień podciągi

substr_compare - Binarne bezpieczne porównanie dwóch ciągów z przesunięcia do długości znaków
substr_count - Policz liczbę wystąpień podciągów
substr_replace - zamień tekst w części ciągu
substr - Zwraca część ciągu
trim - Usuwa białe znaki (lub inne znaki) z początku i końca ciągu
ucfirst - Tworzy wielką literę pierwszego znaku w łańcuchu
ucwords - Wielką literą jest pierwszy znak każdego słowa w ciągu
vfprintf - Napisz sformatowany ciąg do strumienia
vprintf - wyprowadza sformatowany ciąg
vsprintf - Zwraca sformatowany ciąg
wordwrap - zawija ciąg do podanej liczby znaków